

E-Commerce Platform Architecture

From CRUD to Scalable System

LUWI Mafuleti

April 18, 2026

Contents

1	Project Vision	3
2	Actors (System Roles)	3
2.1	Core (Now)	3
2.2	Future Integration	3
3	System Features	3
3.1	Core Features (Now)	3
3.2	Future Features	3
4	Database Design (MLD)	3
4.1	Core Tables (Now)	3
4.2	Planned Tables (Future)	4
5	Relationships (MCD)	4
5.1	Core Relationships	4
5.2	Future Relationships	4
6	Business Logic Considerations	4
6.1	Core (Now)	4
6.2	Future Enhancements	5
7	Order Lifecycle	5
8	Application Pages	5
8.1	Public Pages	5
8.2	User Pages	5
8.3	Admin Pages	5
8.4	Future Pages	5
9	Security Considerations	6
9.1	Core	6
9.2	Future	6
10	Performance Considerations	6

11 Development Roadmap	6
12 Technical Specification Layer	7
12.1 API Design	7
12.1.1 Core Endpoints (Now)	7
12.1.2 Example Request / Response	7
12.2 Validation Rules	7
12.2.1 Core (Now)	7
12.2.2 Future Enhancements	8
13 Business Constraints & Edge Cases	8
13.1 Core Constraints (Now)	8
13.2 Critical Edge Cases	8
13.3 Future Constraints	8
14 System Boundaries	8
14.1 Handled by Laravel (Core)	8
14.2 External Services (Future)	8
15 Deployment Strategy	8
15.1 Environments	8
15.2 Configuration	9
15.3 Database Management	9
15.4 Future Improvements	9
16 Conclusion	9

Project Vision

The objective is to transform a basic CRUD system into a scalable e-commerce platform.

The system must support product browsing, cart management, order processing, and administrative control while being extensible for future integrations.

Actors (System Roles)

Core (Now)

- Guest – browse products
- User – purchase products
- Admin – manage products, orders and users

Future Integration

- Vendor – multi-seller marketplace

System Features

Core Features (Now)

- Product listing and details
- Add to cart
- Checkout system
- Order creation
- Authentication (login / register)

Future Features

- Payment gateway integration
- Product reviews and ratings
- Email notifications
- Advanced search and filtering
- Multi-vendor support

Database Design (MLD)

Core Tables (Now)

Table	Description
users	id, name, email, password, role
products	id, name, price, description, stock, image, category _{id}
categories	id, name
orders	id, user _{id} , total _{price} , status, created _{at}
order _{items}	id, order _{id} , product _{id} , quantity, price
cart	id, user _{id}

<i>cart_items</i>	<i>id, cart_id, product_id, quantity</i>
-----------------------------	--

Planned Tables (Future)

Table	Purpose
<i>product_iimages</i>	multiple images per product
<i>product_variants</i>	SKU, size, colour, stock per variant, discount price
<i>reviews</i>	<i>user_id, product_id, rating, comment, created_at</i>
<i>payments</i>	<i>order_id, method, status, transaction_id, amount</i>
<i>addresses</i>	<i>user_id, line1, line2, city, state, zip, country</i>
<i>vendors</i>	<i>id, name, contact_info</i>
<i>vendor_products</i>	<i>vendor_id, product_id</i>

Relationships (MCD)

Core Relationships

- User **has many** Orders
- Order **has many** OrderItems
- Product **belongs to** Category
- Cart **belongs to** User
- Cart **has many** CartItems
- OrderItem **references** Product

Future Relationships

- Product **has many** Images
- Product **has many** Variants
- User **has many** Addresses
- Product **has many** Reviews
- Order **has one** Payment
- Vendor **has many** Products

Conceptual Diagram:

User < Orders < OrderItems > Product > Category

< Cart < CartItems > Product

< Addresses

Business Logic Considerations

Core (Now)

- Add product to cart
- Create order from cart
- Reduce stock after purchase
- Prevent adding out-of-stock items

Future Enhancements

- Restore stock on cancellation
- Merge guest cart after login
- Payment verification and refunds
- Low-stock alerts
- Review moderation

Order Lifecycle

- pending
- paid
- processing
- shipped
- delivered
- cancelled

Application Pages

Public Pages

- Home
- Product list
- Product details
- Login / Register

User Pages

- Cart
- Checkout
- Order history
- Profile & address management

Admin Pages

- Dashboard
- Manage Products (CRUD, images, variants)
- Manage Orders (status updates, refunds)
- Manage Users & Roles
- Review moderation

Future Pages

- Vendor portal
- Advanced search results
- Wishlist

Security Considerations

Core

- CSRF protection (Laravel default)
- Password hashing (bcrypt)
- Input validation & sanitisation

Future

- Role-based access control (admin vs. user)
- Authorization policies for resource actions
- Rate limiting on sensitive endpoints
- Secure payment webhook handling

Performance Considerations

- Avoid N+1 queries – eager load relationships
- Index frequently queried columns (e.g., `product_name`, `category_id`)
- Cache product listings / popular items
- Pagination for large result sets

Development Roadmap

Phase	Tasks
Phase 1	• Core tables & migrations • Product listing & details • Cart (session + persistent) • Order creation (basic statuses) • Basic authentication
Phase 2	• Admin panel (CRUD for products, categories, orders) • Role-based middleware • Stock management (decrease / restore) • Email notifications (order confirmation, password reset)
Phase 3	• Payment integration (Stripe / PayPal) • Product images & variants • Search & filtering (name, category, price) • Reviews & ratings • Low-stock alerts

Phase 4

- Multi-vendor support
 - API layer (Laravel Sanctum)
 - Front-end SPA (Vue/React) optional
 - Advanced analytics dashboard
-

Technical Specification Layer

API Design

Core Endpoints (Now)

Endpoint	Description
GET /products	Retrieve all products (paginated)
GET /products/id	Retrieve a single product
POST /cart/add	Add product to cart
GET /cart	View cart items
POST /checkout	Create an order from cart
POST /login	Authenticate user
POST /register	Create new user account

Example Request / Response

POST /checkout

Request:

```
{
  "user_id": 1,
  "cart_items": [
    {"product_id": 2, "quantity": 1}
  ]
}
```

Response:

```
{
  "order_id": 15,
  "status": "pending",
  "total_price": 250
}
```

Validation Rules

Core (Now)

- Product: name (required), price (numeric), stock (integer)
- User: email (unique, valid), password (min 6 chars)
- Order: must contain at least one item
- Cart: quantity must be > 0

Future Enhancements

- Strong password rules
- Email verification
- Product SKU uniqueness

Business Constraints & Edge Cases

Core Constraints (Now)

- Prevent checkout if cart is empty
- Prevent adding products with zero stock
- Lock product price at time of order

Critical Edge Cases

- **Stock Change During Checkout:** re-validate stock before order creation
- **Race Conditions:** use database transactions to avoid overselling
- **Duplicate Orders:** prevent double submission (idempotency)

Future Constraints

- Payment failure handling (retry / cancel)
- Refund logic and stock restoration
- Fraud detection mechanisms

System Boundaries

Handled by Laravel (Core)

- Routing and controllers
- Business logic (orders, cart, products)
- Authentication and sessions
- Database interaction (Eloquent ORM)

External Services (Future)

- Payment gateway (Stripe / PayPal)
- Email service (notifications, verification)
- Search engine (Elasticsearch / Algolia)
- CDN for product images

Deployment Strategy

Environments

- Local: development and testing
- Staging: pre-production validation
- Production: live system

Configuration

- Use `.env` for environment variables
- Separate database credentials per environment
- Debug mode disabled in production

Database Management

- Use migrations for schema control
- Run migrations during deployment
- Seed initial data when necessary

Future Improvements

- CI/CD pipeline (GitHub Actions)
- Automated testing before deployment
- Containerization (Docker)

Conclusion

The project evolves from a simple CRUD application into a structured, production-ready e-commerce platform.

By separating core functionality from future extensions, the architecture remains manageable while providing a clear path for scaling, security hardening, and feature growth.